
UNIVERSIDADE E MODELAGEM DE SISTEMAS
ALGORITMOS E PROGRAMAÇÃO ORIENTADA A OBJETOS II
(APO2)

DOCUMENTO DE DIRETRIZES
ARQUITETURAIS E
DECISÕES ESTRATÉGICAS DE DESIGN

Sistema de Gestão de Biblioteca com Indexação MARC21

Estudante Orientado: Engenharia de Software / Ciência da Computação
Mentoria Técnica: Ambiente de Orientação APO2

17 de maio de 2026

Sumário

1	Introdução e Visão Geral do Sistema	3
1.1	Stack Tecnológica Obrigatória e Justificativa de Engenharia	3
2	Engenharia de Requisitos Completa	3
2.1	Requisitos Funcionais (RF)	3
2.2	Requisitos Não Funcionais (RNF)	5
3	Estratégias Lógicas e Decisões de Design de Orientação a Objetos	5
3.1	A Polêmica dos Atores: Por que Rejeitamos a Herança de Classes para Papéis de Usuário	5
3.2	O Cenário Correto para Aplicação de Herança	6
3.3	Gerenciamento de Memória Eficiente: Lazy Loading via DAOs	7
4	Arquitetura de Software: Separação entre Servlets e DAOs	7
4.1	A Camada Controller: Os Servlets como Guardiões de Contexto	7
4.2	A Camada Model-Data: Os DAOs como Camada Pura de Persistência	8
5	Modelagem Física do Banco de Dados Relacional (MySQL 8.0)	9
5.1	A Estratégia Híbrida Chave-Valor para o MARC21	9

1 Introdução e Visão Geral do Sistema

Este documento técnico tem como objetivo formalizar, consolidar e detalhar todas as decisões arquiteturais, escolhas de design orientado a objetos e estratégias de modelagem de dados desenvolvidas para o projeto final da disciplina de **Algoritmos e Programação Orientada a Objetos II (APO2)**.

O sistema proposto consiste em uma plataforma corporativa para a **Gestão de Biblioteca**, estruturada sob a rígida separação de conceitos do padrão **MVC (Model-View-Controller)**. O grande diferencial técnico deste sistema reside na obrigatoriedade de trabalhar com o padrão internacional de catalogação bibliográfica **MARC21** utilizando bases puramente relacionais, o que exige soluções estratégicas para evitar o acoplamento de código e a degradação de performance do banco de dados.

1.1 Stack Tecnológica Obrigatória e Justificativa de Engenharia

As seguintes tecnologias foram estabelecidas como premissas de projeto, tendo suas escolhas justificadas sob o ponto de vista de arquitetura de software web multiusuário:

- **Plataforma Jakarta EE (J2EE) & Apache Tomcat 9.0+:** Uso nativo da API de `HttpServlet` para o gerenciamento de requisições e controle de estado através de `HttpSession`. O Tomcat atua como um container leve de Servlets, ideal para a execução do MVC sem a necessidade de um servidor de aplicação completo (EJB).
- **Banco de Dados MySQL Server 8.0+:** Escolha fundamentada na robustez transacional (suporte nativo ao motor ACID InnoDB), controle estrito de chaves estrangeiras e recursos avançados de indexação de strings, fundamentais para buscas no padrão MARC21.
- **JavaServer Pages (JSP) & JSTL:** Utilizado estritamente na camada *View* para renderização estrutural do HTML dinâmico no servidor, utilizando Taglibs para evitar completamente a inserção de *scriptlets* (código Java bruto dentro do HTML).
- **jQuery & AJAX com Respostas JSON:** Estratégia central para garantir uma experiência de usuário (UX) fluida e assíncrona. Toda a comunicação de dados entre a interface e o servidor ocorre por meio de rotas parametrizadas que devolvem exclusivamente objetos serializados em **JSON**, eliminando *page reloads* completos.
- **Bootstrap 5.0:** Framework utilitário CSS responsável pela responsividade e padronização visual da interface de usuário em dispositivos móveis e desktops.

2 Engenharia de Requisitos Completa

Abaixo encontram-se explicitados os requisitos mínimos e avançados do sistema, devidamente catalogados para fins de rastreabilidade durante os testes funcionais via Postman.

2.1 Requisitos Funcionais (RF)

RF1 Landing Page Pública: Interface institucional contendo campos de busca global e exibição dos livros em destaque no acervo.

- RF2 Mecanismo de Autenticação (Login):** Tela protegida para inserção de credenciais, efetuando o direcionamento lógico com base no nível de acesso do usuário.
- RF3 Auto-Cadastro de Usuários:** Formulário público para a criação de novas contas de leitores na plataforma.
- RF4 Recuperação de Credenciais:** Mecanismo para geração de links seguros de redefinição de senhas expiradas ou esquecidas.
- RF5 Validação de Conta por E-mail:** Sistema assíncrono que valida a conta do usuário por meio de um token temporário enviado via protocolo SMTP (utilizando Mailtrap no ambiente de homologação).
- RF6 Atualização de Perfil Autenticado:** Funcionalidade privada que permite a modificação de dados cadastrais (nome, senha e dados de contato) do usuário logado.
- RF7 Busca Avançada MARC21:** Consulta parametrizada ao acervo permitindo filtros por Título (Tag 245), Autor (Tag 100), ISBN (Tag 020) ou tags customizadas de assunto (Tag 650).
- RF8 Gerenciamento de Listas de Leitura:** Permite ao cliente logado salvar obras de interesse em coleções personalizadas para consultas futuras.
- RF9 Solicitação de Empréstimo:** Fluxo de reserva e checagem de viabilidade para locação de um exemplar físico do acervo.
- RF10 Consulta de Histórico Pessoal:** Listagem detalhada de todos os empréstimos ativos, atrasados e finalizados pertencentes ao usuário autenticado.
- RF11 Renovação de Prazo Online:** Extensão do período de locação, condicionada às regras paramétricas do negócio.
- RF12 CRUD de Usuários (Administrativo):** Painel exclusivo do Administrador (*Manager*) para listagem, bloqueio, desbloqueio e alteração manual de papéis de usuários.
- RF13 Catalogação MARC21 Avançada:** Interface administrativa para cadastro indexado do acervo seguindo convenções bibliotecárias internacionais (Tags, indicadores e subcampos).
- RF14 Controle de Inventário e Estoque:** Gerenciamento quantitativo automatizado do volume de exemplares físicos disponíveis para retirada.
- RF15 Baixa Operacional de Empréstimos:** Módulo interno para registro físico da devolução de livros e cálculo imediato de sanções.
- RF16 Dashboard Analítico Gerencial:** Exibição gráfica e estatística de métricas operacionais, como índices de inadimplência e títulos mais locados.
- RF17 Mecanismo de Recomendação Inteligente (Opcional):** Algoritmo auxiliar focado em sugerir novas leituras com base nas preferências prévias e comportamento histórico do usuário.
- RF18 Parametrização de Regras Globais:** Tela de configuração para o Administrador estipular o valor fixado para a multa diária e o limite de renovações consecutivas.

2.2 Requisitos Não Funcionais (RNF)

- RNF1 Regra de Temporalidade Padrão:** O cálculo padrão para o prazo regulamentar de devolução de qualquer obra física é fixado em exatamente 7 dias corridos a partir da data de retirada.
- RNF2 Notificação Proativa de Vencimento:** O sistema deve disparar automaticamente um e-mail de alerta para o leitor exatamente 24 horas antes do prazo previsto para a devolução.
- RNF3 Bloqueio Automático por Inadimplência:** Se o usuário possuir qualquer livro com status “Atrasado” ou possuir multas financeiras em aberto, o sistema deve bloquear imediatamente a realização de novos empréstimos.
- RNF4 Penalização Financeira Progressiva:** Multas de natureza financeira devem ser geradas de forma cumulativa, automática e diária para cada dia de atraso verificado após a data limite prevista.
- RNF5 Restauração Condicional de Status:** A alteração do estado do usuário de “Bloqueado” para “Ativo” está estritamente condicionada à concomitância da devolução física do item e da quitação integral do débito no caixa.
- RNF6 Responsividade de Interface:** Toda a aplicação web deve se adaptar dinamicamente aos padrões visuais de telas móveis e desktops, garantindo acessibilidade sem quebra de layouts.
- RNF7 Arquitetura Baseada em Mensageria Assíncrona (UX):** Nenhuma operação crítica (como login, buscas ou renovações) pode forçar o recarregamento total da página web, devendo ser gerenciada via jQuery AJAX.

3 Estratégias Lógicas e Decisões de Design de Orientação a Objetos

A transição do modelo mental de requisitos para o código-fonte Java exige a tomada de decisões de design cruciais. Esta seção detalha as estratégias adotadas e as armadilhas de modelagem rejeitadas.

3.1 A Polêmica dos Atores: Por que Rejeitamos a Herança de Classes para Papéis de Usuário

Durante a modelagem do Diagrama de Classes, uma abordagem ingênua sugeriria a criação de uma superclasse `Usuario` e duas subclasses: `LibraryMember` e `Manager`, mimetizando os atores do Caso de Uso. Essa estratégia foi ****expressamente rejeitada**** devido aos seguintes fatores de engenharia de software:

1. **Acoplamento Rígido e Mutabilidade de Papéis:** No ecossistema Java, a herança é estática em tempo de execução. Se um usuário do tipo `LibraryMember` for promovido a `Manager`, o sistema não conseguiria converter a instância do objeto nativamente na memória, exigindo rotinas destrutivas no banco de dados (deletar de uma tabela e reinserir em outra).

2. **Redundância Estrutural:** Ambas as classes compartilhariam exatamente os mesmos atributos de identificação e persistência (`id`, `nome`, `email`, `senha`).

A Estratégia Adotada: Aplicamos o padrão arquitetural **RBAC (Role-Based Access Control)** com Composição. Centralizamos o modelo em uma única entidade rica chamada `Usuario`, cujo comportamento e nível de permissão são governados por um atributo do tipo Enumerador (`Enum`).

```
1 public enum TipoUsuario {
2     ADMIN,    // Equivale ao papel 'Manager'
3     CLIENTE   // Equivale ao papel 'Library_Member'
4 }
5
6 public class Usuario {
7     private int id;
8     private String nome;
9     private String email;
10    private String senha;
11    private boolean ativo;
12    private TipoUsuario tipoUsuario; // Composicao
13
14    // Metodo de autoverificacao rico (Domain Logic)
15    public boolean ehAdministrador() {
16        return this.tipoUsuario == TipoUsuario.ADMIN;
17    }
18
19    // Getters e Setters tipados...
20 }
```

Listing 1: Implementação Estratégica do Controle de Acesso por Papéis (RBAC)

3.2 O Cenário Correto para Aplicação de Herança

A herança por generalização foi reservada exclusivamente para o domínio do acervo da biblioteca. Itens físicos e digitais possuem ciclos de vida imutáveis e atributos inteiramente distintos, configurando o cenário perfeito para o princípio de substituição de Liskov (LSP).

```
1 public abstract class ItemAcervo {
2     protected int id;
3     protected String titulo;
4     protected int anoPublicacao;
5
6     public abstract void processarRegraEmprestimo();
7 }
8
9 public class LivroFisico extends ItemAcervo {
10    private String isbn;
11    private String localizacaoEstante;
12    private int quantidadeEstoque;
13
14    @Override
15    public void processarRegraEmprestimo() {
16        // Reduz estoque fisico e aplica prazo de 7 dias
17    }
18 }
19
20 public class EBook extends ItemAcervo {
```

```

21     private String urlDownload;
22     private double tamanhoArquivoMB;
23
24     @Override
25     public void processarRegraEmprestimo() {
26         // Gera link criptografado temporario sem impactar estoque
27         fisico
28     }

```

Listing 2: Uso Correto de Herança no Domínio do Acervo

3.3 Gerenciamento de Memória Eficiente: Lazy Loading via DAOs

Para mitigar riscos de estouro de memória (`OutOfMemoryError`) no Apache Tomcat sob acessos simultâneos, rejeitou-se o modelo de carregamento antecipado (*Eager Loading*). Ao efetuar o login, o objeto `Usuario` alocado na `HttpSession` armazena apenas dados básicos de identificação.

Dados históricos massivos (como a listagem de empréstimos ocorridos nos últimos anos) são carregados de forma estritamente tardia (**Lazy Loading**), sendo invocados de forma externa por métodos que aceitam o objeto de escopo completo.

```

1 public class EmprestimoDAO extends BaseDAO {
2     // Passamos o objeto rico como escopo para garantir extensibilidade
3     futura
4     public List<Emprestimo> listarHistoricoPorUsuario(Usuario usuario) {
5         String sql = "SELECT * FROM empréstimos WHERE usuario_id = ?";
6         List<Emprestimo> lista = new ArrayList<>();
7         // Execucao do JDBC usando usuario.getId()...
8         return lista;
9     }

```

Listing 3: Assinatura Orientada a Objetos para Busca Dinâmica

4 Arquitetura de Software: Separação entre Servlets e DAOs

Para garantir um código com **Alta Coesão** e **Baixo Acoplamento**, dividimos a lógica do ecossistema Web seguindo a metáfora culinária clássica de engenharia: os Servlets atuam como os “Garçons” (atendimento ao cliente e regras de tráfego) e os DAOs atuam como os “Cozinheiros” (manipulação direta dos dados brutos).

4.1 A Camada Controller: Os Servlets como Guardiões de Contexto

Os Servlets estendem de `HttpServlet` e são os únicos responsáveis por entender o protocolo HTTP. Eles interceptam requisições AJAX, gerenciam a `HttpSession`, validam privilégios de acesso e devolvem Strings formatadas em JSON. **É terminantemente proibido inserir qualquer linha de comando SQL dentro de um Servlet.**

```

1 @WebServlet("/api/livros/cadastrar")
2 public class LivroServlet extends HttpServlet {
3
4     @Override
5     protected void doPost(HttpServletRequest request,
6         HttpServletResponse response)
7         throws ServletException, IOException {
8
9         // Recupera o Usuario alocado na memoria RAM do servidor
10        Usuario usuarioLogado = (Usuario) request.getSession().
11        getAttribute("usuarioAutenticado");
12
13        // Barreira de Seguranca Baseada em Papel (RBAC)
14        if (usuarioLogado == null || !usuarioLogado.ehAdministrador()) {
15            response.setStatus(HttpServletResponse.SC_FORBIDDEN); //
16            HTTP 403
17            response.getWriter().write("{\"erro\": \"Acesso restrito a
18            administradores.\"}");
19            return;
20        }
21
22        // Processamento assincrono de dados do MARC21
23        String titulo = request.getParameter("titulo");
24        String autor = request.getParameter("autor");
25        String isbn = request.getParameter("isbn");
26
27        Livro novoLivro = new Livro(titulo, autor, isbn);
28        LivroDAO livroDAO = new LivroDAO();
29
30        boolean sucesso = livroDAO.inserir(novoLivro);
31
32        response.setContentType("application/json");
33        response.setCharacterEncoding("UTF-8");
34        if (sucesso) {
35            response.getWriter().write("{\"sucesso\": true, \"mensagem
36            \": \"Obra catalogada!\"}");
37        } else {
38            response.setStatus(HttpServletResponse.
39            SC_INTERNAL_SERVER_ERROR);
40            response.getWriter().write("{\"erro\": \"Falha interna ao
41            persistir dados.\"}");
42        }
43    }
44 }

```

Listing 4: Exemplo de Servlet atuando como Controlador e barreira de segurança

4.2 A Camada Model-Data: Os DAOs como Camada Pura de Persistência

As classes DAO (*Data Access Objects*) servem unicamente para abrir conexões com o MySQL, injetar parâmetros em instruções *PreparedStatement* (protegendo a aplicação contra ataques de *SQL Injection*) e converter coleções relacionais (*ResultSet*) de volta para coleções de objetos Java. O DAO é completamente agnóstico em relação à Web; ele não sabe o que é uma sessão HTTP, um navegador ou uma string JSON.

5 Modelagem Física do Banco de Dados Relacional (MySQL 8.0)

O maior desafio de modelagem deste projeto é acomodar a especificação de catalogação internacional **MARC21**. O MARC21 prevê centenas de tags variáveis (campos de dados identificados por números de 3 dígitos, como 020 para ISBN, 100 para Autor, 245 para Título, 650 para Assunto).

5.1 A Estratégia Híbrida Chave-Valor para o MARC21

Criar uma coluna na tabela para cada tag existente do MARC21 geraria uma tabela absurdamente larga, esparsa (cheia de valores nulos) e inflexível.

A Decisão Estratégica: Adotamos um modelo híbrido. Mantemos colunas fixas estruturadas na tabela **livros** apenas para os dados essenciais e imutáveis de busca indexada de alta performance (Título, Autor, ISBN). Para comportar qualquer outra tag variável exigida no cadastro bibliotecário, criamos uma tabela satélite chamada **campos_marc**, que mapeia os dados no padrão **Chave-Valor** sob um relacionamento 1:N com a tabela principal.

Abaixo encontra-se o script de definição física de dados (DDL) limpo e otimizado para o MySQL 8.0.

```
1  -- Remocao previa para evitar conflitos em ambientes de teste
2  DROP TABLE IF EXISTS campos_marc;
3  DROP TABLE IF EXISTS emprestimos;
4  DROP TABLE IF EXISTS livros;
5  DROP TABLE IF EXISTS usuarios;
6
7  -- 1. TABELA DE USUARIOS UNIFICADA (PADRAO RBAC)
8  CREATE TABLE usuarios (
9      id INT AUTO_INCREMENT,
10     nome VARCHAR(100) NOT NULL,
11     email VARCHAR(100) NOT NULL,
12     senha VARCHAR(255) NOT NULL, -- Destinada a armazenar HASH (BCrypt/
    SHA-256)
13     token_ativacao VARCHAR(100) NULL,
14     ativo BOOLEAN DEFAULT FALSE,
15     tipo_usuario ENUM('ADMIN', 'CLIENTE') NOT NULL,
16     CONSTRAINT pk_usuarios PRIMARY KEY (id),
17     CONSTRAINT uk_usuarios_email UNIQUE (email)
18 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
19
20 -- 2. TABELA DE LIVROS (CAMPOS CRITICOS E FIXOS DO MARC21)
21 CREATE TABLE livros (
22     id INT AUTO_INCREMENT,
23     isbn VARCHAR(20) NOT NULL, -- Equivale nativamente a Tag MARC 020
24     titulo VARCHAR(255) NOT NULL, -- Equivale nativamente a Tag MARC
    245
25     autor VARCHAR(150) NOT NULL, -- Equivale nativamente a Tag MARC
    100
26     qtd_total INT NOT NULL DEFAULT 1,
27     qtd_disponivel INT NOT NULL DEFAULT 1,
28     CONSTRAINT pk_livros PRIMARY KEY (id),
29     CONSTRAINT uk_livros_isbn UNIQUE (isbn),
30     CONSTRAINT chk_estoque_positivo CHECK (qtd_disponivel <= qtd_total)
```

```

31 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
32
33 -- 3. TABELA DE FLEXIBILIDADE MARC21 (PADRAO CHAVE-VALOR 1:N)
34 -- Permite indexar infinitas tags variaveis (Ex: 650-Assunto, 490-Series
    ) sem alterar a estrutura do banco
35 CREATE TABLE campos_marc (
36     id INT AUTO_INCREMENT,
37     livro_id INT NOT NULL,
38     tag VARCHAR(3) NOT NULL, -- Armazena strings numericas como '650',
    '500', '700'
39     valor TEXT NOT NULL,      -- Conteudo textual indexado da tag
40     CONSTRAINT pk_campos_marc PRIMARY KEY (id),
41     CONSTRAINT fk_marc_livros FOREIGN KEY (livro_id)
42         REFERENCES livros(id) ON DELETE CASCADE
43 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
44
45 -- 4. TABELA CORE DE NEGOCIO: EMPRESTIMOS (RELACIONAMENTO N:M NATIVO)
46 CREATE TABLE emprestimos (
47     id INT AUTO_INCREMENT,
48     usuario_id INT NOT NULL,
49     livro_id INT NOT NULL,
50     data_emprestimo DATE NOT NULL,
51     data_devolucao_prevista DATE NOT NULL, -- Calculada via Java (
    data_emprestimo + 7 dias)
52     data_devolucao_efetiva DATE NULL,      -- Mantida nula enquanto o
    item estiver em posse do leitor
53     multa_acumulada DECIMAL(10,2) DEFAULT 0.00,
54     renovacoes_decorrentes INT DEFAULT 0,
55     CONSTRAINT pk_emprestimos PRIMARY KEY (id),
56     CONSTRAINT fk_emprestimos_usuarios FOREIGN KEY (usuario_id)
57         REFERENCES usuarios(id) ON DELETE RESTRICT,
58     CONSTRAINT fk_emprestimos_livros FOREIGN KEY (livro_id)
59         REFERENCES livros(id) ON DELETE RESTRICT
60 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
61
62 -- Criacao estrategica de indices para otimizacao de consultas e buscas
    rapidas
63 CREATE INDEX idx_usuarios_login ON usuarios(email, senha);
64 CREATE INDEX idx_livros_busca_rapida ON livros(titulo, autor);
65 CREATE INDEX idx_marc_tag_valor ON campos_marc(tag, valor(50));

```

Listing 5: Script de Criação de Tabelas (DDL) para o MySQL 8.0